

Sake Import Checker - 포트폴리오 가이드

목적: 이 프로젝트의 설계 이유, 구현 방식, 서비스 연결 구조를 초보자도 이해할 수 있게 설명합니다.

목차

1. 프로젝트 개요
2. 전체 아키텍처
3. 서비스 연결 구조
4. 핵심 기능 구현
5. 데이터베이스 설계
6. Smart Update 시스템
7. 설계 결정의 이유
8. 핵심 코드 설명

1. 프로젝트 개요

무엇을 하는 서비스인가?

사케/와인 라벨 사진 → 한국 수입 이력 검색

사용자가 텔레그램으로 사케나 와인 라벨 사진을 보내면, AI가 라벨을 분석하고 한국 수입 데이터베이스에서 해당 제품을 찾아 다음 정보를 알려줍니다:

- 제품명 (한글/영문)
- 제조사/와이너리
- 수입사
- 수입 금액 및 물량
- 단가

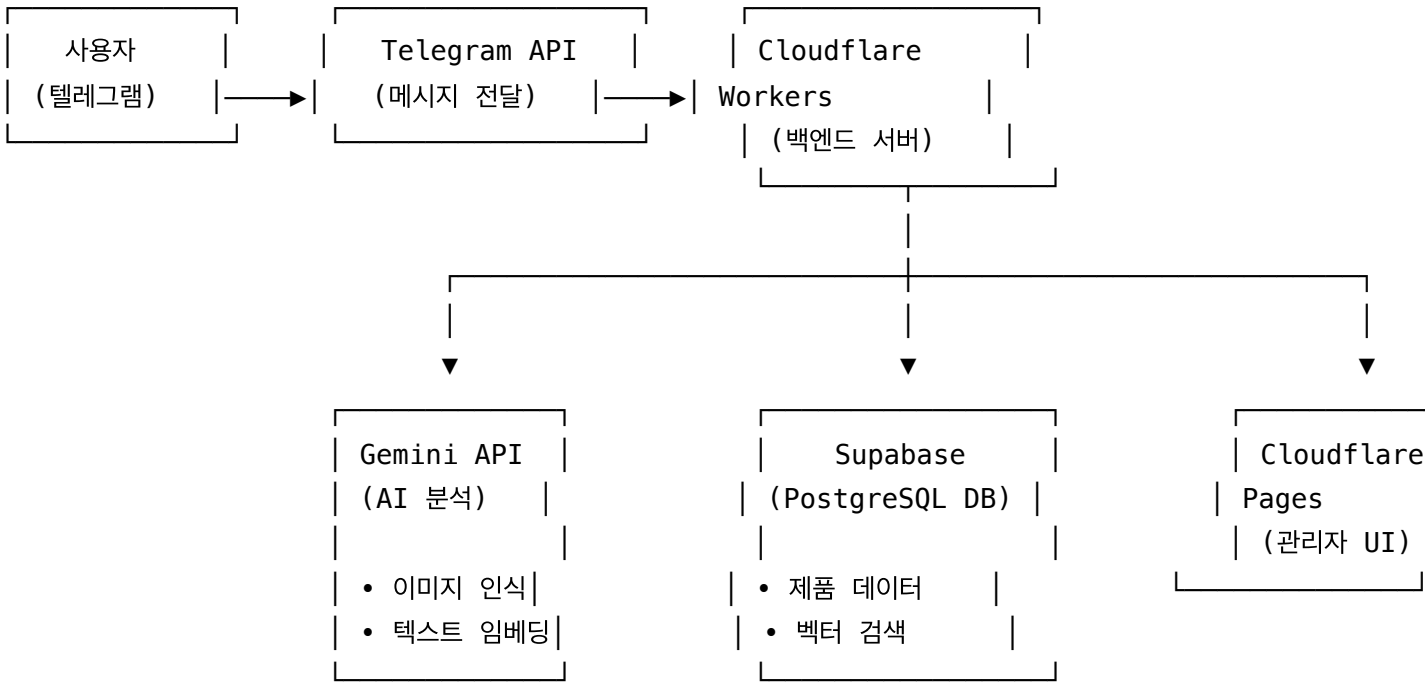
왜 이 서비스를 만들었나?

1. **문제:** 사케/와인 라벨은 일본어/영어로 되어 있어 한국 소비자가 읽기 어려움

- 2. **해결**: 사진만 찍으면 AI가 자동으로 인식하고 한국어 정보 제공
- 3. **가치**: 수입 이력/가격 정보로 제품의 인기도와 적정 가격 판단 가능

2. 전체 아키텍처

시스템 구성도



사용 기술 스택

영역	기술	역할
백엔드	Cloudflare Workers	서버리스 API 서버
프레임워크	Hono (TypeScript)	라우팅, 미들웨어
데이터베이스	Supabase (PostgreSQL)	제품 데이터 저장
벡터 검색	pgvector (HNSW)	유사 제품 검색
AI	Google Gemini	이미지 인식, 텍스트 임베딩
인터페이스	Telegram Bot API	사용자 대화

영역	기술	역할
관리자 UI	Cloudflare Pages	데이터 업로드 페이지

3. 서비스 연결 구조

3.1 Telegram Bot API 연결

Webhook 방식: 사용자가 메시지를 보내면 Telegram이 우리 서버로 HTTP 요청을 보냄

사용자 → Telegram 앱 → Telegram 서버 → [POST 요청] → 우리 서버

↓

<https://our-worker.workers.dev/telegram-webhook>

왜 Webhook인가?

- **Polling 방식:** 서버가 주기적으로 "새 메시지 있어요?" 물어봄 → 비효율적
- **Webhook 방식:** 메시지가 올 때만 서버가 호출됨 → 효율적, 서버리스에 적합

연결 설정 방법:

```
# Telegram에 우리 서버 주소 등록 (한 번만 실행)
curl -X POST "https://api.telegram.org/bot<TOKEN>/setWebhook" \
  -d "url=https://sake-import-checker.workers.dev/telegram-webhook"
```

코드에서의 구현 (backend/src/index.ts):

```
import { Hono } from 'hono';

const app = new Hono<{ Bindings: Env }>();

// Telegram이 이 URL로 POST 요청을 보냄
app.post('/telegram-webhook', handleTelegramWebhook);
```

3.2 Cloudflare Workers 연결

서버리스란?

- 전통적 서버: 24시간 켜져 있는 컴퓨터 (비용 발생)
- 서버리스: 요청이 올 때만 실행되는 함수 (사용한 만큼만 비용)

왜 Cloudflare Workers인가?

1. 무료 티어 넉넉함: 일 10만 요청 무료
2. 전 세계 빠른 응답: CDN 엣지에서 실행
3. 간단한 배포: wrangler deploy 한 줄로 배포

환경변수 설정 (민감한 정보는 Secret으로):

```
# wrangler.toml에 직접 쓰면 안 되는 정보들
wrangler secret put TELEGRAM_BOT_TOKEN
wrangler secret put GEMINI_API_KEY
wrangler secret put SUPABASE_URL
wrangler secret put SUPABASE_KEY
wrangler secret put ADMIN_PASSWORD
```

3.3 Supabase 연결

Supabase란?

- Firebase의 오픈소스 대안
- PostgreSQL 데이터베이스 + REST API + 인증 등 제공
- 우리는 주로 데이터베이스와 **RPC** 함수 사용

연결 방식: Supabase JavaScript 클라이언트

```
import { createClient } from '@supabase/supabase-js';

// 환경변수에서 URL과 API 키 가져옴
const supabase = createClient(env.SUPABASE_URL, env.SUPABASE_KEY);

// 데이터 조회 예시
const { data, error } = await supabase
  .from('sake_imports')
  .select('*')
  .eq('category', 'Sake');
```

RPC 함수란?

- PostgreSQL에 저장된 함수를 원격으로 호출
- 복잡한 쿼리를 서버에서 한 번에 처리 (네트워크 효율)

```
// 벡터 검색 RPC 호출
const { data } = await supabase.rpc('search_products', {
  query_embedding: [0.1, 0.2, ...], // 768차원 벡터
  match_count: 50,
  threshold: 0.5
});
```

3.4 Gemini API 연결

Gemini란?

- Google의 AI 모델 (GPT의 구글 버전)
- Vision: 이미지 분석
- Embedding: 텍스트를 숫자 벡터로 변환

연결 방식: REST API 호출

```
// Gemini Vision API 호출
const response = await fetch(
  `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateCon
  {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      contents: [{
        parts: [
          { text: "이 사케 라벨에서 브랜드명을 추출해주세요" },
          { inline_data: { mime_type: "image/jpeg", data: base64Image } }
        ]
      }]
    })
  }
);
```

3.5 서비스 간 데이터 흐름

1. 사용자가 사진 전송
 - ↳ Telegram API가 Webhook으로 우리 서버 호출
2. Cloudflare Workers가 요청 처리
 - ↳ Telegram API에서 사진 파일 다운로드
 - ↳ Gemini Vision으로 라벨 분석
 - ↳ Gemini Embedding으로 검색용 벡터 생성
 - ↳ Supabase에서 벡터 검색
3. 결과를 Telegram API로 전송
 - ↳ 사용자에게 메시지 도착

4. 핵심 기능 구현

4.1 Triple Search (3중 검색)

왜 3단계 검색인가?

단순 키워드 검색의 문제:

- "닷사이 23" vs "獺祭 二割三分" → 같은 제품인데 매칭 안 됨
- 오타나 표기 차이에 취약

해결책: 의미 기반 검색 (Semantic Search)

- 1단계: Vision Extraction (이미지 → 텍스트)
 - ↳ AI가 라벨에서 브랜드명, 숫자, 제조사 추출
- 2단계: Vector Search (텍스트 → 유사 제품)
 - ↳ 텍스트를 벡터로 변환 후 유사도 검색
- 3단계: AI Verification (후보 → 최종 확인)
 - ↳ 상위 후보들을 원본 사진과 비교 검증

4.2 벡터 검색이란?

개념 설명

"닷사이 23" → [0.12, 0.45, 0.78, ...] (768개 숫자)
"獺祭 二割三分" → [0.11, 0.44, 0.79, ...] (비슷한 벡터!)
"산토리 위스키" → [0.89, 0.21, 0.03, ...] (다른 벡터)

- 비슷한 의미의 텍스트 → 비슷한 벡터
- 벡터 간 거리(코사인 유사도)로 유사도 측정
- 0.0 = 완전히 다름, 1.0 = 완전히 같음

PostgreSQL에서 벡터 검색 (pgvector)

```
-- 벡터 검색 쿼리 (코사인 거리)
SELECT *, 1 - (name_embedding <=> query_embedding) AS similarity
FROM sake_imports
WHERE 1 - (name_embedding <=> query_embedding) > 0.5
ORDER BY similarity DESC
LIMIT 50;
```

HNSW 인덱스

- 벡터 검색을 빠르게 하는 인덱스 알고리즘
- 56,000개 행을 매번 전체 비교하면 느림
- HNSW로 관련 있는 것들만 빠르게 찾을 수 있음

4.3 High Confidence Skip

문제: AI 검증(3단계)이 시간과 비용 소모

해결: 확실한 경우 검증 생략

```
// search.ts에서 신뢰도 판단
if (similarity >= 0.82) {
  // 매우 높은 유사도 → 바로 반환
  return { found: true, confidence: similarity * 100, product: topProduct };
}

if (similarity >= 0.75 && (similarity - secondSimilarity) > 0.15) {
  // 높은 유사도 + 2등과 격차 큼 → 바로 반환
  return { found: true, confidence: similarity * 100, product: topProduct };
}

// 그 외: AI 검증 필요
const verified = await verifyMatch(env, base64Image, candidates.slice(0, 3));
```

4.4 제품 타입별 검색 최적화

사케와 와인은 라벨 특성이 다름

구분	사케	와인
핵심 정보	브랜드명, 숫자 (23, 39 등)	와이너리, 빈티지 년도
라벨 언어	일본어 + 영어	영어/불어 등
검색 우선순위	숫자 매칭	와이너리 매칭

코드에서의 구현 (search.ts):


```

function prioritizeByMetadata(products: Product[], extracted: ExtractedLabelInfo) {
  if (extracted.productType === 'Wine') {
    // 와인: 와이너리 매칭 점수 100점
    return products.sort((a, b) => {
      const aScore = a.exporter?.includes(extracted.exporterEnglish) ? 100 : 0;
      const bScore = b.exporter?.includes(extracted.exporterEnglish) ? 100 : 0;
      return bScore - aScore;
    });
  } else {
    // 사케: 숫자 포함 여부로 우선순위
    return products.sort((a, b) => {
      const aHasNumber = extracted.numbers.some(n => a.reported_product_name.includes(n));
      const bHasNumber = extracted.numbers.some(n => b.reported_product_name.includes(n));
      return (bHasNumber ? 1 : 0) - (aHasNumber ? 1 : 0);
    });
  }
}

```

5. 데이터베이스 설계

5.1 메인 테이블 구조

```
CREATE TABLE sake_imports (  
  id BIGSERIAL PRIMARY KEY,  
  
  -- 제품 정보  
  reported_product_name TEXT NOT NULL, -- "닷사이 23 (Dassai 23)"  
  category TEXT, -- Sake, Wine, Spirits, etc.  
  exporter TEXT, -- 제조사/와이너리  
  origin_country TEXT, -- 원산지  
  raw_importer_name TEXT, -- 수입사  
  
  -- 수입 데이터  
  value NUMERIC, -- 수입 금액 (USD)  
  volume NUMERIC, -- 수입 물량 (kg)  
  unit_price NUMERIC, -- 단가 (USD/kg)  
  
  -- 검색용 벡터 (768차원)  
  name_embedding vector(768),  
  
  -- 메타데이터  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW()  
);
```

5.2 인덱스 설계

```
-- 1. 벡터 검색 인덱스 (HNSW)
CREATE INDEX idx_sake_imports_embedding
ON sake_imports
USING hnsw (name_embedding vector_cosine_ops)
WITH (m = 24, ef_construction = 128);

-- 2. 카테고리 필터링 인덱스
CREATE INDEX idx_sake_imports_category
ON sake_imports (category);

-- 3. Smart Update용 복합 인덱스
CREATE INDEX idx_sake_imports_composite_coalesce
ON sake_imports (
    reported_product_name,
    COALESCE(exporter, ''),
    COALESCE(origin_country, ''),
    COALESCE(raw_importer_name, '')
);
```

5.3 주요 RPC 함수

벡터 검색 함수:

```

CREATE OR REPLACE FUNCTION search_products(
    query_embedding vector(768),
    match_count INT DEFAULT 50,
    threshold FLOAT DEFAULT 0.5,
    category_filter TEXT DEFAULT NULL
)
RETURNS TABLE (
    id BIGINT,
    reported_product_name TEXT,
    category TEXT,
    exporter TEXT,
    origin_country TEXT,
    raw_importer_name TEXT,
    value NUMERIC,
    volume NUMERIC,
    unit_price NUMERIC,
    similarity FLOAT
)
AS $$
BEGIN
    -- hnsw.ef_search 설정 (검색 정확도)
    SET LOCAL hnsw.ef_search = 100;

    RETURN QUERY
    SELECT
        s.id, s.reported_product_name, s.category,
        s.exporter, s.origin_country, s.raw_importer_name,
        s.value, s.volume, s.unit_price,
        (1 - (s.name_embedding <=> query_embedding))::FLOAT AS similarity
    FROM sake_imports s
    WHERE
        s.name_embedding IS NOT NULL
        AND (category_filter IS NULL OR s.category = category_filter)
        AND (1 - (s.name_embedding <=> query_embedding)) > threshold
    ORDER BY s.name_embedding <=> query_embedding
    LIMIT match_count;
END;
$$ LANGUAGE plpgsql;

```

6. Smart Update 시스템

6.1 문제 상황

전체 초기화 방식의 문제점:

- 56,000개 제품 → 임베딩 생성에 30분+ 소요
- API 비용 발생 (Gemini Embedding 호출)
- 매번 전체 재업로드 비효율적

6.2 Smart Update 개념

기존 제품: 수치만 업데이트 (임베딩 유지)

신규 제품: 임베딩 생성 후 추가

4-필드 복합 키로 기존/신규 판단:

name + exporter + origin_country + raw_importer_name
= 고유한 제품 식별

6.3 타임아웃 문제 해결

문제: 대량 UPDATE 시 statement timeout 발생

해결 과정 (7번의 시행착오):

시도	방법	결과
#1	복합 인덱스 생성	41,400개 후 실패
#2	병렬 배치 처리	46,620개 후 실패
#3	RPC 함수 (IS NOT DISTINCT FROM)	20개 후 실패
#4	COALESCE 인덱스	10,320개 후 실패
#5	순차 처리	20,950개 후 실패
#6	배치 축소	매우 불안정
#7	자동 재시도 + 지수 백오프	성공

최종 해결책: 실패해도 자동으로 재시도

```
// upload.js - fetchWithRetry 함수
async function fetchWithRetry(url, options, maxRetries = 5) {
  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      const response = await fetch(url, options);
      if (response.ok) return response;

      if (response.status >= 500) {
        throw new Error(`Server error: ${response.status}`);
      }
      return response;
    } catch (error) {
      if (attempt < maxRetries - 1) {
        // 지수 백오프: 2초 → 4초 → 8초 → 16초 → 32초
        const delay = 2000 * Math.pow(2, attempt);
        await new Promise(r => setTimeout(r, delay));
        continue;
      }
      throw error;
    }
  }
}
```

지수 백오프란?

- 실패할 때마다 대기 시간을 2배로 증가
- 서버가 과부하일 때 회복 시간을 줌
- 무한 재시도 방지 (최대 5회)

7. 설계 결정의 이유

7.1 왜 Cloudflare Workers인가?

대안	장점	단점
AWS Lambda	성숙한 생태계	비용, Cold Start

대안	장점	단점
Vercel	Next.js 친화적	일반 API에 과함
Cloudflare Workers	무료 티어, 빠른 응답	일부 Node.js 기능 제한

선택 이유: 무료로 충분, 전 세계 엣지에서 빠른 응답

7.2 왜 Supabase인가?

대안	장점	단점
Firebase	실시간 DB	SQL 아님, 벡터 검색 불가
PlanetScale	MySQL 호환	벡터 검색 불가
Supabase	PostgreSQL + pgvector	무료 티어 제한

선택 이유: 벡터 검색(pgvector) 필수, PostgreSQL의 유연성

7.3 왜 Gemini인가?

대안	장점	단점
OpenAI GPT-4V	최고 성능	비용 높음
Claude Vision	좋은 성능	비용 높음
Gemini	무료 티어, 좋은 성능	Rate Limit

선택 이유: 무료 티어로 개발/테스트 가능, Vision + Embedding 모두 제공

7.4 왜 텔레그램인가?

대안	장점	단점
카카오톡	한국 사용자 많음	개발 비용, 승인 필요
라인	아시아 사용자	복잡한 API
텔레그램	무료, 간단한 API	한국 사용자 적음

선택 이유: 빠른 개발, 무료, 봇 API 우수

7.5 왜 벡터 검색인가?

전통적 검색의 한계:

-- 키워드 검색: 정확히 일치해야 함

```
SELECT * FROM products WHERE name LIKE '%땃사이%';
```

-- "獺祭" 검색 안 됨!

벡터 검색의 장점:

-- 의미 기반 검색: 비슷한 의미면 찾아냄

```
SELECT * FROM products
```

```
WHERE 1 - (embedding <=> query_embedding) > 0.5;
```

-- "땃사이", "獺祭", "Dassai" 모두 찾음!

8. 핵심 코드 설명

8.1 메인 엔트리포인트 (index.ts)

```
import { Hono } from 'hono';
import { cors } from 'hono/cors';

// Hono 앱 생성 (Express와 유사한 웹 프레임워크)
const app = new Hono<{ Bindings: Env }>();

// CORS 설정: 어떤 도메인에서 API 호출 가능한지
app.use('*', cors({
  origin: ['https://sake-admin.pages.dev', 'http://localhost:8788'],
  allowMethods: ['GET', 'POST', 'OPTIONS'],
  allowHeaders: ['Content-Type', 'Authorization'],
})));

// 라우트 등록
app.get('/health', handleHealth); // 서버 상태 확인
app.post('/telegram-webhook', handleTelegramWebhook); // 텔레그램 메시지 처리
app.post('/admin/upload-init', handleInitUpload); // DB 초기화
app.post('/admin/upload-chunk', handleUploadChunk); // 데이터 업로드
app.get('/admin/stats', handleStats); // 통계 조회

// Cloudflare Workers 엔트리포인트
export default app;
```

8.2 텔레그램 웹훅 핸들러 (telegram.ts)

```
export async function handleTelegramWebhook(c: Context<{ Bindings: Env }>) {
  const update = await c.req.json();

  // 사진 메시지인지 확인
  if (update.message?.photo) {
    // waitUntil: 응답 후에도 백그라운드에서 계속 처리
    // (텔레그램 타임아웃 방지)
    c.executionCtx.waitUntil(handlePhotoMessage(c.env, update.message));
  } else if (update.message?.text) {
    c.executionCtx.waitUntil(handleTextMessage(c.env, update.message));
  }

  // 텔레그램에 즉시 OK 응답 (처리는 백그라운드에서)
  return c.json({ ok: true });
}

async function handlePhotoMessage(env: Env, message: TelegramMessage) {
  const chatId = message.chat.id;
  const messageId = message.message_id;

  // 가장 큰 사진 선택 (Telegram은 여러 해상도 제공)
  const photos = message.photo;
  const bestPhoto = photos
    .filter(p => (p.file_size || 0) <= 5 * 1024 * 1024) // 5MB 이하
    .sort((a, b) => b.width - a.width)[0]; // 가장 큰 것

  // "분석 중..." 메시지 전송
  await sendMessage(env, chatId, '🔍 라벨을 분석하고 있습니다...', {
    reply_to_message_id: messageId
  });

  try {
    // 사진 URL 가져오기
    const fileUrl = await getFileUrl(env, bestPhoto.file_id);

    // 검색 실행 (핵심 로직)
    const result = await searchProduct(env, fileUrl);

    // 결과 포매팅 후 전송
    const responseText = formatSearchResult(result);
    await sendMessage(env, chatId, responseText, {
```

```
        reply_to_message_id: messageId
    });
} catch (error) {
    await sendMessage(env, chatId, '오류가 발생했습니다. 다시 시도해주세요.', {
        reply_to_message_id: messageId
    });
}
}
```

8.3 검색 파이프라인 (search.ts)

```
export async function searchProduct(env: Env, imageUrl: string): Promise<SearchResult>

// 1. 이미지 다운로드 및 Base64 변환
const response = await fetch(imageUrl);
const arrayBuffer = await response.arrayBuffer();
const base64Image = arrayBufferToBase64(arrayBuffer);

// 2. Gemini Vision으로 라벨 정보 추출
const extracted = await extractLabelInfo(env, base64Image);

if (extracted.errorType === 'RATE_LIMIT') {
  return { found: false, message: 'AI 사용량 초과. 잠시 후 다시 시도해주세요.' };
}

// 3. 검색 텍스트 생성 (제품 타입에 따라 다름)
let searchText: string;
if (extracted.productType === 'Wine') {
  // 와인: 와이너리 우선
  searchText = [
    extracted.exporterEnglish, // 와이너리 (가장 중요)
    extracted.region,
    extracted.grapeVariety,
    extracted.vintage,
    extracted.brand
  ].filter(Boolean).join(' ');
} else {
  // 사케: 브랜드 + 숫자 중심
  searchText = [
    extracted.brand,
    extracted.brandKorean,
    extracted.brandEnglish,
    extracted.exporterEnglish,
    extracted.rawText
  ].filter(Boolean).join(' ');
}

// 4. 텍스트 → 벡터 변환
const embedding = await getEmbedding(env, searchText);

// 5. 벡터 검색 (Supabase RPC)
const candidates = await vectorSearch(env, embedding, 50, 0.5);
```

```

if (candidates.length === 0) {
  return { found: false, message: '데이터베이스에서 일치하는 제품을 찾지 못했습니다.' };
}

// 6. 메타데이터 기반 재정렬
const prioritized = prioritizeByMetadata(candidates, extracted);

const topProduct = prioritized[0];
const similarity = topProduct.similarity || 0;

// 7. 신뢰도 판단
if (similarity >= 0.82) {
  // 매우 높은 유사도 → 바로 반환
  return {
    found: true,
    confidence: Math.round(similarity * 100),
    product: topProduct,
    extractedInfo: extracted
  };
}

if (similarity >= 0.75) {
  const secondSimilarity = prioritized[1]?.similarity || 0;
  if (similarity - secondSimilarity > 0.15) {
    // 2등과 큰 격차 → 바로 반환
    return {
      found: true,
      confidence: Math.round(similarity * 100),
      product: topProduct,
      extractedInfo: extracted
    };
  }
}

// 8. AI 검증 (신뢰도 낮은 경우)
const verification = await verifyMatch(env, base64Image, prioritized.slice(0, 3));

if (verification.matchedIndex > 0 && verification.confidence >= 70) {
  return {
    found: true,
    confidence: verification.confidence,
    product: prioritized[verification.matchedIndex - 1],
  };
}

```

```
        extractedInfo: extracted
    };
}

// 9. 불확실한 경우 후보와 함께 반환
return {
    found: false,
    confidence: verification.confidence,
    candidates: prioritized.slice(0, 3),
    message: '확실하지 않습니다. 다음 제품들이 유사합니다.',
    extractedInfo: extracted
};
}
```

8.4 Gemini Vision 분석 (gemini.ts)

```
export async function extractLabelInfo(
  env: Env,
  base64Image: string
): Promise<ExtractedLabelInfo> {

  // 프롬프트: AI에게 무엇을 추출할지 지시
  const prompt = `
당신은 주류 라벨 분석 전문가입니다.
이 이미지에서 다음 정보를 추출해주세요:

1. productType: "Sake" 또는 "Wine"
2. brand: 브랜드명 (원문)
3. brandKorean: 한글 브랜드명 (있으면)
4. brandEnglish: 영문 브랜드명
5. exporterEnglish: 제조사/와이너리 (영문)
6. numbers: 라벨에 보이는 숫자들 (예: ["23", "39"])
7. volume: 용량 (예: "720ml")
8. confidence: 추출 신뢰도 (50-80)

와인인 경우 추가로:
9. region: 생산지역
10. grapeVariety: 포도 품종
11. vintage: 생산연도

JSON 형식으로만 응답해주세요.
`;

  // Gemini API 호출
  const response = await fetch(
    `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateC
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        contents: [{
          parts: [
            { text: prompt },
            { inline_data: { mime_type: 'image/jpeg', data: base64Image } }
          ]
        }],
        generationConfig: {
```

```

        temperature: 0.1, // 낮은 온도 = 일관된 출력
        maxOutputTokens: 1024
    }
})
}
);

const data = await response.json();

// 응답에서 JSON 파싱
const text = data.candidates[0].content.parts[0].text;
const jsonMatch = text.match(/\{[\s\S]*\}/);
const result = JSON.parse(jsonMatch[0]);

return result as ExtractedLabelInfo;
}

```


8.5 관리자 업로드 핸들러 (admin.ts)

```
export async function handleUploadChunk(c: Context<{ Bindings: Env }>) {
  // 인증 확인
  const authHeader = c.req.header('Authorization');
  if (authHeader !== `Bearer ${c.env.ADMIN_PASSWORD}`) {
    return c.json({ error: 'Unauthorized' }, 401);
  }

  const { data } = await c.req.json();

  // 1. 데이터 정제 (컬럼명 공백 제거)
  const cleanedData = data.map(row => {
    const cleaned = {};
    for (const key in row) {
      cleaned[key.trim()] = row[key];
    }
    return cleaned;
  });

  // 2. 기존 제품 조회 (4-필드 복합 키)
  const names = cleanedData.map(d => d.displayName);
  const { data: existing } = await supabase
    .from('sake_imports')
    .select('reported_product_name, exporter, origin_country, raw_importer_name')
    .in('reported_product_name', names);

  // 3. 기존/신규 분리
  const existingMap = new Map();
  for (const p of existing || []) {
    const key = `${p.reported_product_name}|${p.exporter}|${p.origin_country}|${p.raw_i
    existingMap.set(key, true);
  }

  const toUpdate = [];
  const toInsert = [];

  for (const item of cleanedData) {
    const key = `${item.displayName}|${item.Exporter}|${item['Origin Country']}|${item[
    if (existingMap.has(key)) {
      toUpdate.push(item);
    } else {
      toInsert.push(item);
    }
  }
}
```

```
}  
}
```

// 4. 기존 제품: RPC로 일괄 UPDATE (임베딩 유지)

```
if (toUpdate.length > 0) {  
  await supabase.rpc('bulk_update_sake_imports', {  
    updates: toUpdate.map(row => ({  
      name: row.displayName,  
      exporter: row.Exporter,  
      origin: row['Origin Country'],  
      importer: row['Raw Importer Name'],  
      value: row.Value,  
      volume: row.Volume,  
      unit_price: row['Unit Price']  
    })))  
  });  
}
```

// 5. 신규 제품: 임베딩 생성 후 INSERT

```
if (toInsert.length > 0) {  
  const texts = toInsert.map(item => item.embeddingText);  
  const embeddings = await getBatchEmbeddings(c.env, texts);  
  
  const rows = toInsert.map((item, idx) => ({  
    reported_product_name: item.displayName,  
    category: item.category,  
    exporter: item.Exporter,  
    origin_country: item['Origin Country'],  
    raw_importer_name: item['Raw Importer Name'],  
    value: item.Value,  
    volume: item.Volume,  
    unit_price: item['Unit Price'],  
    name_embedding: embeddings[idx]  
  }));  
  
  await supabase.from('sake_imports').insert(rows);  
}  
  
return c.json({  
  ok: true,  
  updated: toUpdate.length,  
  inserted: toInsert.length
```

```
});  
}
```

용어 정리

용어	설명
Webhook	이벤트 발생 시 서버가 다른 서버에 HTTP 요청을 보내는 방식
서버리스	서버 관리 없이 함수 단위로 코드 실행하는 클라우드 서비스
벡터 임베딩	텍스트/이미지를 고차원 숫자 배열로 변환한 것
코사인 유사도	두 벡터 간 각도로 유사도 측정 (0~1)
HNSW	고속 벡터 검색 알고리즘 (Hierarchical Navigable Small World)
RPC	Remote Procedure Call, 원격 서버의 함수를 호출하는 방식
지수 백오프	재시도 간격을 지수적으로 늘리는 에러 복구 전략
CORS	Cross-Origin Resource Sharing, 다른 도메인에서 API 호출 허용 설정
Cold Start	서버리스에서 첫 요청 시 초기화로 인한 지연

마무리

이 프로젝트는 다음 기술들의 조합으로 구성되어 있습니다:

- 1. AI 이미지 인식 (Gemini Vision) - 라벨 텍스트 추출
- 2. 의미 기반 검색 (pgvector + Embeddings) - 유사 제품 검색
- 3. 서버리스 아키텍처 (Cloudflare Workers) - 비용 효율적인 API
- 4. 메시지 인터페이스 (Telegram Bot) - 사용자 접점
- 5. 스마트 데이터터 관리 (Smart Update) - 효율적인 대량 업로드

각 기술은 *****왜 이것인가?*****라는 질문에 명확한 답을 가지고 선택되었으며,
전체 시스템은 **실용성과 비용 효율성**을 최우선으로 설계되었습니다.